Table 8.3.1. Execution Report of Test Case TC-001

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**
Test Case ID:	TC-001
Testing Tools Used:	Android Studio, JUnit 4, Jetpack Compose UI Test
Execution Steps:	

Test Execution Records:

#	Tester	Test Date	Test Stage	Actual Result	Status	Defect	Correction
1	Tariq	11/06/2025	Unit	The "Start Session" button was visible and tapping it invoked the onStart callback, starting the monitoring session	Pass	None	null by N/A

Execution Summary:	TC-001 passed. StartScreen correctly shows the "Start Session" button and the click handler starts a session without errors

Acknowledgment: Generated from the CapStone process management system ©2026

Table 8.3.2. Execution Report of Test Case TC-004

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**						
Test Case ID:	TC-004						
Testing Tools Used:	Android Studio, JUnit 4, Jetpack Compose UI Test						
Execution Steps:	1	Open SessionScreen and tap the "End Session" button once.					
Test Execution Records:							
#	**Tester**	**Test Date**	**Test Stage**	**Actual Result**	**Status**	**Defect**	**Correction**
1	Tariq	11/06/2025	Unit	"End Session" button was visible and tapping it invoked the onEnd callback, ending the current session	Pass	N/A	
Execution Summary:	TC-004 passed. The session can be ended from SessionScreen and the app returns cleanly without crashes						
Acknowledgment: Generated from the CapStone process management system ©2026							

Table 8.3.3. Execution Report of Test Case TC-006

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**	
Test Case ID:	TC-006	
Testing Tools Used:	Android Studio, JUnit 4, Jetpack Compose UI Test	
Execution Steps:	1	On SessionScreen, tap the "Notify" chip twice and observe the dialog text each time

Test Execution Records:

#	Tester	Test Date	Test Stage	Actual Result	Status	Defect	Correction
1	Tariq	11/06/2025	Unit	First tap opened a dialog showing "Notification sent" and "Notify tapped 1 time."; second tap showed "Notify tapped 2 times." as expected	Pass	N/A	null by N/A

Execution Summary:	TC-006 passed. Notify chip correctly opens the dialog and maintains the tap count across multiple presses

Acknowledgment: Generated from the CapStone process management system ©2026

Table 8.3.4. Execution Report of Test Case TC-011

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**	
Test Case ID:	TC-011	
Testing Tools Used:	Android Studio, JUnit 4, Jetpack Compose UI Test	
Execution Steps:	1	On SessionScreen, tap the "Sound" chip, then select "Turn off sound" in the dialog.

Test Execution Records:

#	Tester	Test Date	Test Stage	Actual Result	Status	Defect	Correction
1	Tariq	11/10/2025	Unit	Tapping "Sound" opened the sound dialog ("Playing until you turn it off"); choosing "Turn off sound" stopped the sound and closed the dialog	Pass		

Execution Summary:	TC-011 passed. Sound chip dialog works correctly and the user can turn off the alert sound without issues

Acknowledgment: Generated from the CapStone process management system ©2026

Table 8.3.5. Execution Report of Test Case TC-012

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**	
Test Case ID:	TC-012	
Testing Tools Used:	Android Studio, JUnit 4, Jetpack Compose UI Test	
Execution Steps:	1	On SessionScreen, tap the "Emergency" card once.

Test Execution Records:

#	Tester	Test Date	Test Stage	Actual Result	Status	Defect	Correction
1	Tariq	11/16/2025	Unit	The "Emergency" card was visible and responded to the tap without any crashes or UI errors, confirming the handler is wired	Pass		

Execution Summary:	TC-012 passed. Emergency card is clickable and correctly connected to its tap handler

Acknowledgment: Generated from the CapStone process management system ©2026

Table 8.3.6. Execution Report of Test Case TC-013

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**						
Test Case ID:	TC-013						
Testing Tools Used:	Android Studio, JUnit 4, Mockito						
Execution Steps:	1	Tap Emergency call					
Test Execution Records:							
#	**Tester**	**Test Date**	**Test Stage**	**Actual Result**	**Status**	**Defect**	**Correction**
1	Tariq	11/19/2025	Unit	ACTION_CALL intent with data tel:9513034883 and passed it to	Pass		
Execution Summary:	TC-013 passed. Emergency call helper correctly launches a phone call to the configured emergency number						
Acknowledgment: Generated from the CapStone process management system ©2026							

Table 8.3.7. Execution Report of Test Case TC-014

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**	
Test Case ID:	TC-014	
Testing Tools Used:	JUnit 4	
Execution Steps:	1	Open FaceLogicTest
	2	Run computeImageSize_rotation0_keepsSize()
	3	ImageSizeCalculator.compute(rotation = 0, width = 640, height = 480)
	4	Verify the returned width is 640
	5	Verify the returned height is 480

Test Execution Records:

#	Tester	Test Date	Test Stage	Actual Result	Status	Defect	Correction
1	Tariq	02/20/2026	Unit	The method returned width 640 and height 480 when rotation was 0	Pass	None	null by None

Execution Summary:	Verified that when rotation is 0, the image size remains unchanged.

Acknowledgment: Generated from the CapStone process management system ©2026

Table 8.3.8. Execution Report of Test Case TC-016

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**	
Test Case ID:	TC-016	
Testing Tools Used:	JUnit 4, Kotlin unit test framework, MockK	
Execution Steps:	1	Open FaceLogicTest.
	2	Run evaluator_triggersDrowsy_afterTimeThreshold()
	3	Create a FaceStateEvaluator with a drowsy callback counter.
	4	Enable monitoring.
	5	Evaluate one face with both eyes open.
	6	Evaluate closed-eye frames as time increases.
	7	Let the closed-eye duration pass the drowsy threshold.
	8	Verify that the drowsy callback count becomes 1.

Test Execution Records:

#	Tester	Test Date	Test Stage	Actual Result	Status	Defect	Correction

Execution Summary:	Verified that the evaluator correctly triggers a drowsy alert after eye closure lasts longer than the allowed threshold.

Acknowledgment: Generated from the CapStone process management system ©2026

Table 8.3.9. Execution Report of Test Case TC-017

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**	
Test Case ID:	TC-017	
Testing Tools Used:	JUnit 4, Kotlin unit test framework	
Execution Steps:	1	Run computeImageSize_rotation90_swapsSize()
	2	Call computeImageSize with rotation 90, width 640, and height 480
	3	Check that the returned width is 480
	4	Check that the returned height is 640

Test Execution Records:

#	Tester	Test Date	Test Stage	Actual Result	Status	Defect	Correction

Execution Summary:	Verified that when the image rotation is 90 degrees, computeImageSize correctly swaps the width and height

Acknowledgment: Generated from the CapStone process management system ©2026

Table 8.3.10. Execution Report of Test Case TC-019

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**	
Test Case ID:	TC-019	
Testing Tools Used:	JUnit 4, Kotlin unit test framework	
Execution Steps:	1	Open FaceLogicTest
	2	Run computeImageSize_rotation270_swapsSize()
	3	Call computeImageSize with rotation 270, width 640, and height 480
	4	Check that the returned width is 480
	5	Check that the returned height is 640

Test Execution Records:

#	Tester	Test Date	Test Stage	Actual Result	Status	Defect	Correction

Execution Summary:	Verified that when the image rotation is 270 degrees, computeImageSize correctly swaps the width and height.

Acknowledgment: Generated from the CapStone process management system ©2026

Table 8.3.11. Execution Report of Test Case TC-020

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**	
Test Case ID:	TC-020	
Testing Tools Used:	JUnit 4, Kotlin unit test framework, MockK	
Execution Steps:	1	Run evaluator_doesNotTriggerDrowsy_beforeThreshold()
	2	Create a FaceStateEvaluator with a drowsy callback counter.
	3	Enable monitoring.
	4	Evaluate one face with both eyes open.
	5	Evaluate closed-eye frames as time increases.
	6	Keep the elapsed closed-eye time below the drowsy threshold.
	7	Verify that the drowsy callback count remains 0.

Test Execution Records:

#	Tester	Test Date	Test Stage	Actual Result	Status	Defect	Correction

Execution Summary:	Verified that the evaluator does not trigger a drowsy alert when eye closure has not lasted long enough to cross the threshold.

Acknowledgment: Generated from the CapStone process management system ©2026

Table 8.3.12. Execution Report of Test Case TC-021

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**	
Test Case ID:	TC-021	
Testing Tools Used:	JUnit 4, Kotlin unit test framework, MockK	
Execution Steps:	1	evaluator_drowsy_firesOnlyOnce_forSameClosure()
	2	Create a FaceStateEvaluator with a drowsy callback counter
	3	Enable monitoring.
	4	Evaluate one face with both eyes open.
	5	Evaluate closed-eye frames until the drowsy threshold is exceeded.
	6	Verify the callback fires the first time.
	7	Keep evaluating closed-eye frames without reopening the eyes.
	8	Verify the callback count stays at 1.

Test Execution Records:

#	Tester	Test Date	Test Stage	Actual Result	Status	Defect	Correction

Execution Summary:	The drowsy callback was triggered once and did not fire again during the same continuous eye closure.

Acknowledgment: Generated from the CapStone process management system ©2026

Table 8.3.13. Execution Report of Test Case TC-022

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**	
Test Case ID:	TC-022	
Testing Tools Used:	JUnit 4, Kotlin unit test framework, MockK	
Execution Steps:	1	Open FaceLogicTest.
	2	Run evaluator_drowsy_canFireAgain_afterEyesReopen()
	3	Create a FaceStateEvaluator with a drowsy callback counter.
	4	Enable monitoring.
	5	Evaluate one face with both eyes open.
	6	Evaluate closed-eye frames until the drowsy threshold is exceeded and the callback fires once.
	7	Evaluate a frame where the eyes reopen to reset the drowsy state.
	8	Evaluate closed-eye frames again until the threshold is exceeded a second time.
	9	Verify the callback count becomes 2

Test Execution Records:

#	Tester	Test Date	Test Stage	Actual Result	Status	Defect	Correction

Execution Summary:	Verified that the evaluator resets after the eyes reopen and can issue a new drowsy alert for a later eye-closure event

Acknowledgment: Generated from the CapStone process management system ©2026

Table 8.3.14. Execution Report of Test Case TC-023

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**	
Test Case ID:	TC-023	
Testing Tools Used:	JUnit 4, Kotlin unit test framework, MockK	
Execution Steps:	1	Open FaceLogicTest.
	2	Run evaluator_triggersDistracted_afterTimeThreshold().
	3	Create a FaceStateEvaluator with a distracted callback counter.
	4	Start calibration and set the calibration target to forward.
	5	Evaluate several forward-facing frames to establish the baseline.
	6	Finish calibration successfully.
	7	Enable monitoring.
	8	Evaluate a face turned far away from the forward position.
	9	Increase time past the distraction threshold and evaluate again.
	10	Verify that the distracted callback count becomes 1.

Test Execution Records:

#	Tester	Test Date	Test Stage	Actual Result	Status	Defect	Correction

Execution Summary:	Verified that the evaluator correctly detects distraction and triggers the callback after the head stays turned away beyond the time threshold.

Acknowledgment: Generated from the CapStone process management system ©2026

Table 8.3.15. Execution Report of Test Case TC-024

Project Name:	**Distracted/Drowsy Driver Detection on Snapdragon Mobile Platform**	
Test Case ID:	TC-024	
Testing Tools Used:	JUnit 4, Kotlin unit test framework, MockK	
Execution Steps:	1	Open FaceLogicTest.
	2	Run evaluator_doesNotTriggerDistracted_beforeThreshold().
	3	Create a FaceStateEvaluator with a distracted callback counter.
	4	Start calibration and set the calibration target to forward.
	5	Evaluate several forward-facing frames to establish the baseline.
	6	Finish calibration successfully.
	7	Enable monitoring.
	8	Evaluate a face turned far away from the forward position.
	9	Increase time, but keep it below the distraction threshold.
	10	Evaluate again and verify that the distracted callback count remains 0.

Test Execution Records:

#	Tester	Test Date	Test Stage	Actual Result	Status	Defect	Correction

Execution Summary:	Verified that the evaluator does not issue a distracted alert before the distraction time threshold is reached

Acknowledgment: Generated from the CapStone process management system ©2026